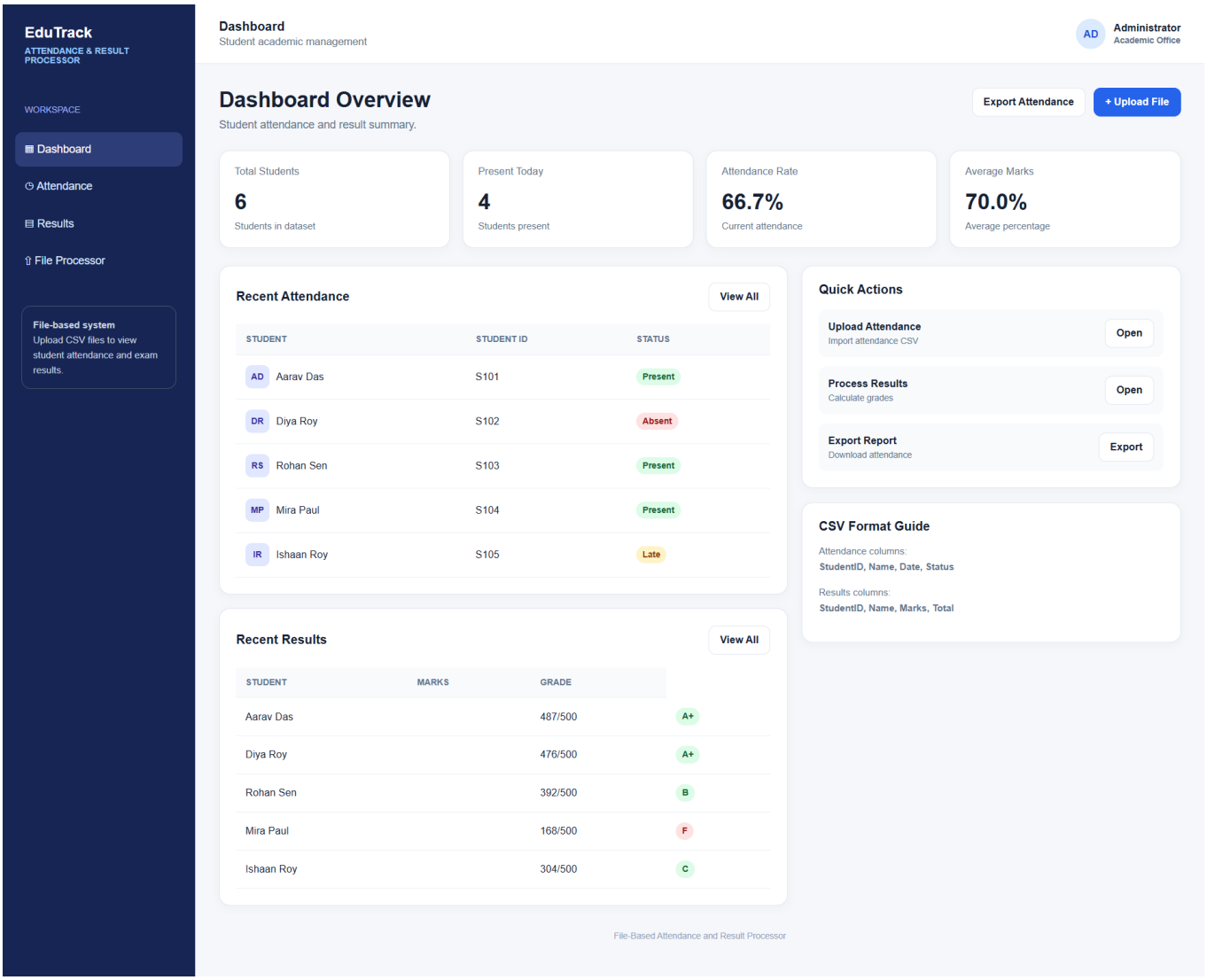
MINI PROJECT REPORT

EDUTRACK

File-Based Attendance & Result Processor

Student Academic Management System



Dashboard Overview

Project Detail	Entry
Prepared by	Arpita Das & Team
Class / Department	Department of Computer Science & Engineering
Institution	Budge Budge Institute of Technology (BBIT)
Submission Date	31 October 2026

PROJECT TITLE & ABSTRACT

Project Title

EduTrack: File-Based Attendance and Result Processor — a client-side web mini-project for automated CSV parsing, student attendance management, performance analytics, percentage evaluation and grade assignment.

Abstract

EduTrack is an administrative academic processing workspace designed to streamline student performance management. Educational institutions often maintain separate records for daily attendance and examination marks using flat CSV files. EduTrack brings these workflows into one browser-based interface. It presents attendance records, calculates class statistics, evaluates examination percentages, assigns letter grades from A+ to F, and displays pass/fail status. The supplied implementation uses JavaScript DOM rendering and includes dynamic filtering, full-text search, summary cards and CSV import/export controls.

Introduction

Academic administrative tasks require teachers and department coordinators to reconcile daily attendance rosters with examination score sheets. When records are held in paper registers or disconnected spreadsheets, calculating percentages and mapping grades is time-consuming and may lead to errors. EduTrack provides a consolidated workspace where records can be reviewed, searched and processed with consistent rules.

Problem Statement

Educational staff need a lightweight way to process attendance and examination scores without repeatedly applying spreadsheet formulas or manually searching separate registers. The project addresses record visibility, calculation consistency and quick access to academic summaries.

Objectives

- Provide Dashboard, Attendance and Result Processor modules in one workspace.
- Read student attendance and marks from structured CSV data.
- Calculate attendance metrics and examination percentages automatically.
- Apply the configured A+ to F grade boundaries and pass/fail rule.
- Enable text search across student records.
- Provide CSV export controls for processed records.

EXISTING SYSTEM VS. PROPOSED SYSTEM

Existing Approach	Typical Limitation	EduTrack Proposed Solution
Paper registers	Records are slow to search; statistics require manual counting.	Searchable digital tables and summary metric cards.
Manual spreadsheets	Formula entry and cell editing can cause inconsistent calculations.	JavaScript calculation pipeline applies repeatable rules.
Separate files	Attendance and marks are reviewed in different places.	Unified dashboard with Attendance and Results modules.
Manual grade assignment	Repeated threshold checking is time-consuming.	Automatic percentage and grade evaluation.

Scope of the Project

- Dashboard overview with student count, present-today count, attendance rate and average marks.
- Attendance table with student ID, name, date and status.
- Results table with marks, total, percentage, grade and result.
- Search/filter controls and CSV import/export affordances.
- Sample student dataset for demonstrating the interface.

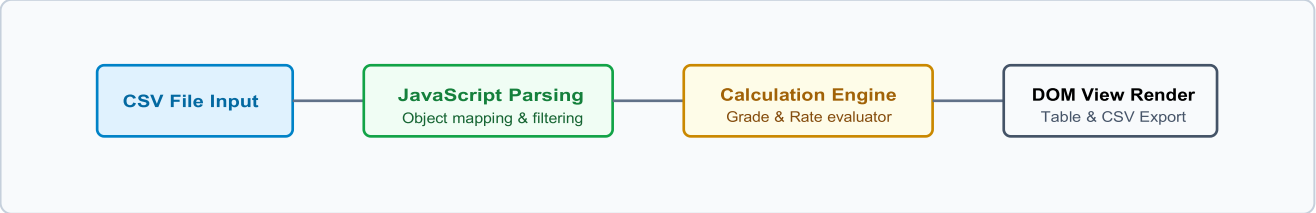
Hardware and Software Requirements

Area	Requirement / project context
User hardware	Standard computer or smartphone capable of running a modern browser.
Operating system	Windows, Android.
Frontend	HTML5, modern CSS (Flexbox/Grid), JavaScript (ES6+).
Browser	Modern browser with JavaScript enabled.
Input/output	CSV files for attendance and examination records.
Development file	JavaScript source referenced in the supplied report: script.js.

SYSTEM ARCHITECTURE & DATA FLOW

The system ingests student records from CSV files, maps the fields into JavaScript objects, applies filtering and calculation logic, and updates the dashboard and tables in the browser.

1. CSV File Input	2. JavaScript Parsing	3. Calculation Engine	4. DOM View Render
Attendance CSV Results CSV	Read rows and map fields StudentID, Name, Date, Status, Marks, Total	Attendance-rate evaluator Percentage calculator Grade / pass-fail rules	Dashboard metric cards Attendance & Results tables CSV export



Data Flow

```
Attendance / Results CSV
          ↓
Read and map records into JavaScript objects
          ↓
Validate fields and prepare datasets
          ↓
┌──────────────────────────┬──────────────────────────┐
│ Attendance processing    │ Result processing        │
│ Search/filter records    │ Marks ÷ Total × 100      │
│ Count attendance status  │ Assign grade & outcome   │
└──────────────────────────┴──────────────────────────┘
          ↓
Update dashboard cards and table rows in the DOM
          ↓
Review records / export processed CSV
```

Main Data Fields

Dataset	Fields shown or implied by the supplied UI
Attendance	StudentID, Name, Date, Status
Results	StudentID, Name, Marks, Total
Derived result	Percentage, Grade, Result status
Dashboard metrics	Total students, Present today, Attendance rate, Average marks

IMPLEMENTATION & CORE SOURCE CODE

The following excerpts are adapted from the supplied JavaScript screenshots to document the core behavior.

Attendance Search and Filtering

The function reads the search box, converts the query to lowercase and retains records where any field contains the query. Matching rows are then rendered in the attendance table.

```
function renderAttendance() {
  let query = (
    document.getElementById("attendanceSearch")?.value || ""
  ).toLowerCase();

  let rows = attendance.filter(student =>
    Object.values(student).some(value =>
      String(value).toLowerCase().includes(query)
    )
  );

  // rows are mapped to table HTML and displayed
  // the record-count label is updated afterwards
}
```

Result Percentage and Grade

```
let percentage = Number(student.Total) > 0
  ? Number(student.Marks) / Number(student.Total) * 100
  : 0;

let studentGrade = grade(percentage);
let resultStatus = studentGrade === "F" ? "Fail" : "Pass";
let badgeClass = studentGrade === "F" ? "fail" : "pass";
```

Grade Scale

Grade	Percentage
A+	90% and above
A	80% and above
B	70% and above
C	60% and above
D	40% and above
F	Below 40%

The grade thresholds and F-to-Fail rule are taken from the supplied EduTrack report and screenshots.

IMPLEMENTATION — SOURCE SCREENSHOTS

The screenshots below preserve the supplied code evidence from the project's JavaScript file.

```
<!-- DASHBOARD -->
<section class="view active" id="dashboard">
    <div class="page-head">
        <div>
            <h1>Dashboard Overview</h1>
            <p>Student attendance and result summary.</p>
        </div>
        <div class="actions">
            <button class="btn" onclick="downloadCSV('attendance')">
                Export Attendance
            </button>
            <button class="btn primary" onclick="showView('files')">
                + Upload File
            </button>
        </div>
    </div>

    <div class="stats">
        <div class="stat">
            <div class="stat-top">Total Students</div>
            <strong id="totalStudents">6</strong>
            <small>Students in dataset</small>
        </div>
        <div class="stat">
            <div class="stat-top">Present Today</div>
            <strong id="presentCount">4</strong>
            <small>Students present</small>
        </div>
        <div class="stat">
            <div class="stat-top">Attendance Rate</div>
            <strong id="attendanceRate">66.7%</strong>
            <small>Current attendance</small>
        </div>
        <div class="stat">
            <div class="stat-top">Average Marks</div>
            <strong id="averageMarks">78.2%</strong>
            <small>Average percentage</small>
        </div>
    </div>
```

Figure 1. Dashboard

```
C: > Users > Arpita Das > Desktop > BBIT > JS script.js > ...
138   }
139
140   // Display and search attendance records
141   function renderAttendance() {
142       let query = (
143           document.getElementById("attendanceSearch")?.value || ""
144       ).toLowerCase();
145
146       let rows = attendance.filter(student =>
147           Object.values(student).some(value =>
148               String(value).toLowerCase().includes(query)
149           )
150       );
151
152       document.getElementById("attendanceTable").innerHTML =
153           rows.map(student => `
154               <tr>
155                   <td>${esc(student.StudentID)}</td>
156                   <td>
157                       <div class="student">
158                           <div class="mini-avatar">
159                               ${esc(initials(student.Name))}
160                           </div>
161                           ${esc(student.Name)}
162                       </div>
163                   </td>
164                   <td>${esc(student.Date)}</td>
165                   <td>${statusBadge(student.Status)}</td>
166               </tr>
167           `).join("") ||
168           `<tr><td colspan="4" class="empty">No matching records.</td></tr>`;
169
170       document.getElementById("attendanceSub").textContent =
171           rows.length + " record(s) shown";
172   }
```

Figure 2. Attendance status.

C: > Users > Arpita Das > Desktop > BBIT > JS script.js > ...

```
// Display and search results
function renderResults() {
    let query = (
        document.getElementById("resultsSearch")?.value || ""
    ).toLowerCase();

    let rows = results.filter(student =>
        Object.values(student).some(value =>
            String(value).toLowerCase().includes(query)
        )
    );

    document.getElementById("resultsTable").innerHTML =
        rows.map(student => {
            let percentage = Number(student.Total) > 0
                ? Number(student.Marks) / Number(student.Total) * 100
                : 0;

            let studentGrade = grade(percentage);
            let resultStatus = studentGrade === "F" ? "Fail" : "Pass";
            let badgeClass = studentGrade === "F" ? "fail" : "pass";

            return `
                <tr>
                    <td>${esc(student.StudentID)}</td>
                    <td>${esc(student.Name)}</td>

                    <td>${esc(student.Marks)}/${esc(student.Total)}</td>
                    <td>${percentage.toFixed(1)}%</td>
                    <td>
                        <span class="badge ${badgeClass}">
                            ${studentGrade}
                        </span>
                    </td>
                    <td>
```

Figure 3. Result status.

C: > Users > Arpita Das > Desktop > BBIT > JS script.js > ...

```
// Export attendance or result records
function downloadCSV(type) {
    let rows;
    let headers;
    let filename;

    if (type === "results") {
        headers = [
            "StudentID", "Name", "Marks",
            "Total", "Percentage", "Grade", "Result"
        ];

        rows = results.map(student => {
            let percentage = Number(student.Total) > 0
                ? Number(student.Marks) / Number(student.Total) * 100
                : 0;

            let studentGrade = grade(percentage);

            return {
                ...student,
                Percentage: percentage.toFixed(1) + "%",
                Grade: studentGrade,
                Result: studentGrade === "F" ? "Fail" : "Pass"
            };
        });

        filename = "student_results.csv";

    } else {
        headers = ["StudentID", "Name", "Date", "Status"];
        rows = attendance;
        filename = "attendance_records.csv";
    }
```

Figure 4. Attendance & Result status.

REPRESENTATIVE SYSTEM SCREENS

The following application outputs show the Dashboard, Attendance and Result Processor modules operating on the sample student dataset.

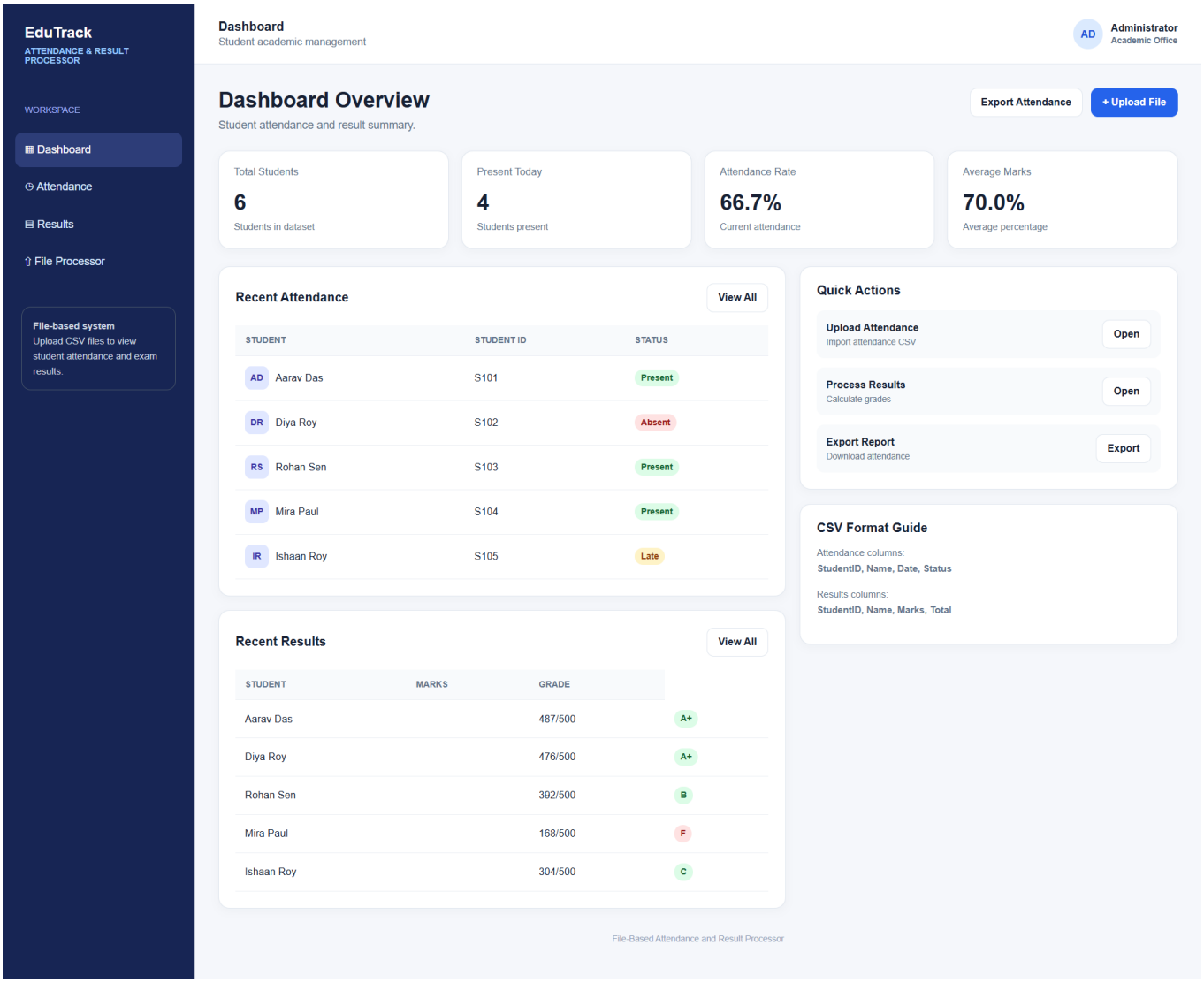


Figure 4. Dashboard Overview: summary cards, recent records and quick actions.

Dashboard Metrics

Metric	Displayed value
Total Students	6
Present Today	4
Attendance Rate	66.7%
Average Marks	70.0%

The dashboard combines high-level indicators with recent attendance and result rows to help an administrator review the sample dataset at a glance.

ATTENDANCE MANAGEMENT & RESULT PROCESSOR

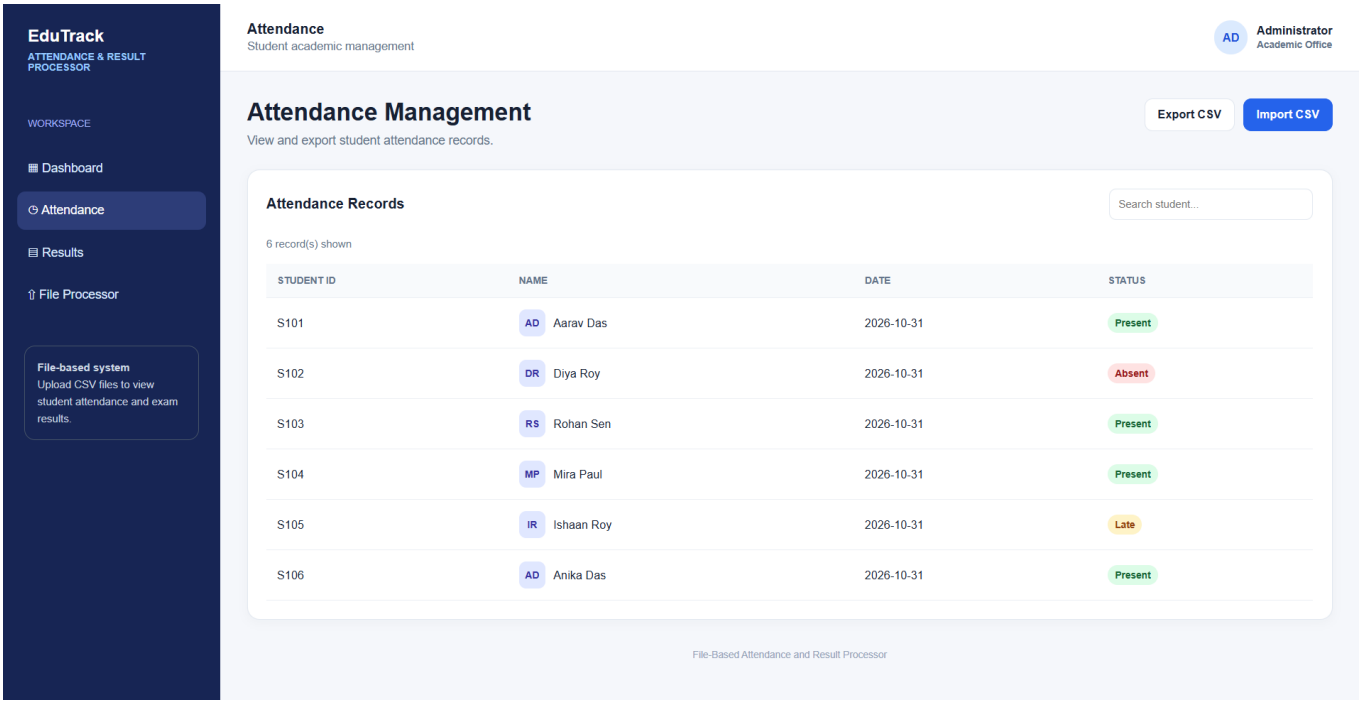


Figure 5. Attendance Management: student IDs, names, dates, statuses and search controls.

Attendance statuses visible in the sample include Present, Absent and Late. The page includes CSV import/export controls and a search field.

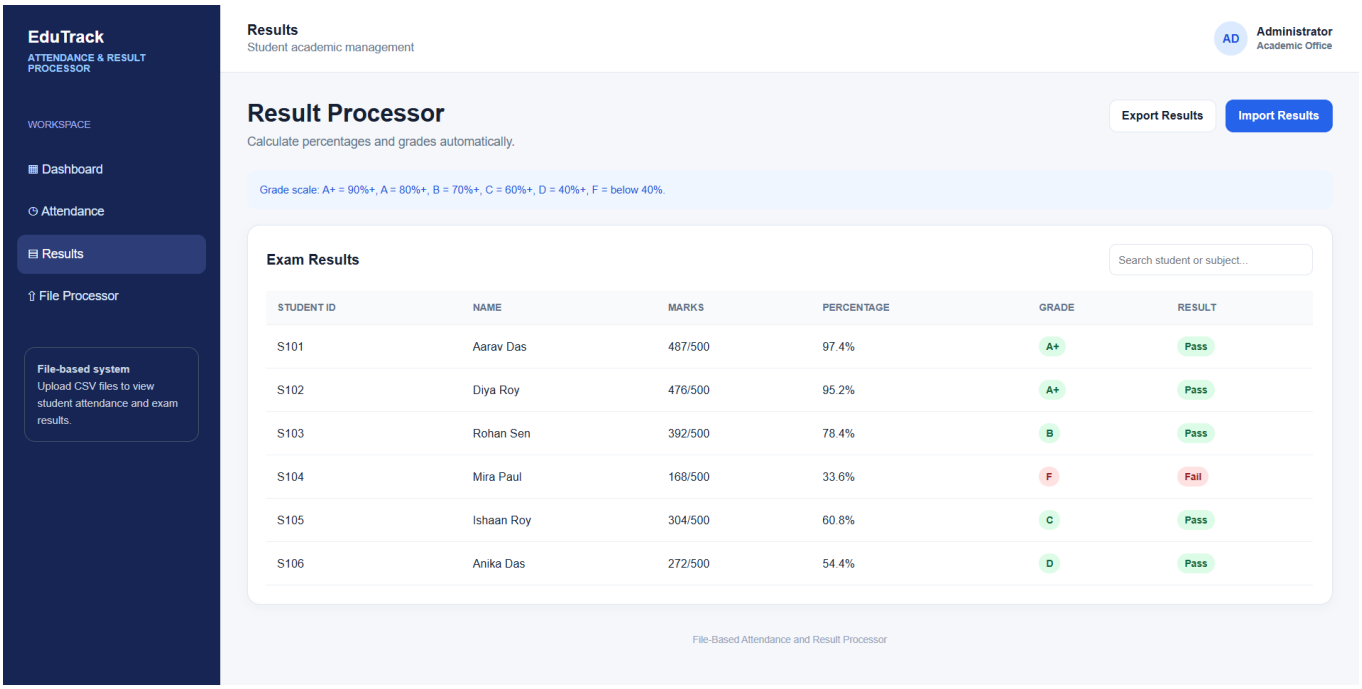


Figure 6. Result Processor: marks, percentage, grade and pass/fail badges.

The Results view displays marks against a total, calculates a percentage to one decimal place and presents the assigned grade and outcome.

TEST CASES AND VERIFICATION RESULTS

Test ID	Test Case / Expected Output	Observed Result	Status
TC-01	Load dashboard with student dataset	Displays 6 students, 4 present and 66.7% attendance.	PASS
TC-02	Process 487/500 marks (Aarav Das)	97.4%, grade A+, result Pass.	PASS
TC-03	Process 168/500 marks (Mira Paul)	33.6%, grade F, result Fail.	PASS
TC-04	Search attendance table	Shown JavaScript filters rows using the query string.	PASS (code review)
TC-05	Export attendance/results to CSV	Export controls are visible in the supplied UI.	Visible; execution not independently verified

Sample Calculation

```
Aarav Das:
Percentage = (487 / 500) × 100 = 97.4%
Grade = A+  →  Result = Pass

Mira Paul:
Percentage = (168 / 500) × 100 = 33.6%
Grade = F   →  Result = Fail
```

Verification Note

The values above are supported by the supplied report and screenshots. The interface and code excerpts were reviewed; file import/export execution, malformed-file handling and long-term persistence were not independently exercised during this report preparation.

Advantages

- Reduces repetitive calculations.
- Improves visibility of attendance and academic performance.
- Makes student records easier to search.
- Keeps attendance and results in one interface.

CONCLUSION, FUTURE SCOPE & REFERENCES

Conclusion

EduTrack provides a clear browser-based workspace for file-based student attendance and examination result processing. The dashboard summarizes the sample dataset, the attendance module presents searchable records, and the result processor calculates percentages, assigns grades and displays pass/fail outcomes. The supplied screenshots and JavaScript excerpts demonstrate the intended interface and core calculation logic.

Future Scope

- Add stronger CSV validation and helpful error messages for missing or invalid fields.
- Support historical term-wise records through a suitable local or server-side data store.
- Generate printable PDF transcripts and student report cards.
- Add performance charts and attendance trend analytics.
- Introduce secure login and role-based access if deployed for multiple users.
- Expand automated tests for grade boundaries, empty totals and malformed CSV files.

References

- EduTrack Mini Project Report, supplied PDF (6 pages).
- Supplied EduTrack Dashboard, Attendance Management and Result Processor screenshots.
- Supplied JavaScript screenshots showing attendance filtering and result calculation.
- MDN Web Docs — File and FileReader interfaces (background reference).
- ECMAScript documentation — array methods and DOM manipulation (background reference).

Report prepared from the user-provided project report, interface screenshots and code screenshots. Where implementation details were not visible in those sources, they have not been stated as verified facts.